SkillHub • ASP.NET Core 8 Web API Architecture & Concepts Guide

Comprehensive Engineering Manual: Role-Based Authentication (RBAC), Entity Framework Core 8, Dependency Injection (DI), DTOs, Model Binding, Middleware Pipeline, builder.Services & REST Controllers

ASP.NET Core 8 | C# 12 / .NET 8 | Clean Architecture | JWT & RBAC | EF Core 8 | Dependency Injection | Middleware Pipeline | Model Binding & DTOs

ASP.NET Core 8 Enterprise Web API: Architecture & Core Concepts Guide

> **Authoritative Technical Engineering Specification & Implementation Manual**
> **SkillHub Enterprise Platform • ASP.NET Core 8 • Clean Architecture • C# 12**

Table of Contents

1. Architectural Overview & Clean Architecture Topology
2. builder.Services & Program.cs (IoC Container & App Bootstrapping)
3. Dependency Injection (DI) & Inversion of Control (IoC)
4. Entity Framework Core 8 (EF Core) Deep Dive
5. Data Transfer Objects (DTOs) & Object Mapping
6. Model Binding & HTTP Parameter Resolution
7. Controllers & RESTful API Orchestration
8. Role-Based Authentication (RBAC) & Claims Authorization
9. HTTP Middleware Pipeline & Error Handling
10. End-to-End Request Flow: Tracing a Request Lifecycle
11. Technical Interview & Viva Voce Master Cheat Sheet

1. Architectural Overview & Clean Architecture Topology

Enterprise ASP.NET Core 8 Web APIs adhere to **Clean Architecture** (also known as Onion or Hexagonal Architecture). The fundamental rule governing Clean Architecture is the **Dependency Inversion Principle**: *dependencies only point inward towards business domain logic; domain models never depend on outer layers (databases, frameworks, UI).*





Layer Responsibilities

LAYER	PRIMARY RESPONSIBILITIES	EXAMPLES IN SKILLHUB
Domain	Enterprise business rules, entities, enums, domain exceptions. Zero framework dependencies.	`User`, `Course`, `Enrollment`, `SupportTicket`, `TicketPriority`
Application	Use case orchestration, interfaces, DTO contracts, AutoMapper profiles, validation.	`ICourseService`, `CourseService`, `CreateCourseRequestDto`, `MappingProfile`
Infrastructure	Database access (EF Core), external APIs, authentication token generation, file storage.	`ApplicationDbContext`, `JwtTokenGenerator`, `SupportService`, BCrypt
API	HTTP entry point, routing, model binding, authorization attributes, middleware pipeline.	`CourseController`, `AuthController`, `GlobalExceptionMiddleware`, `Program.cs`

---

2. builder.Services & Program.cs (IoC Container & App Bootstrapping)

In .NET 8, application startup uses **Top-Level Statements** and the unified `WebApplicationBuilder`. This replaces the legacy `Startup.cs` pattern with a clean, two-phase configuration model: 1. **Phase 1: Service Configuration (`builder.Services`)** — Registering components into the Inversion of Control (IoC) container. 2. **Phase 2: HTTP Request Pipeline Configuration (`app.Use...()`)** — Defining the order of middleware that processes incoming requests.

```
Incoming Process Arguments (args)
           |
           ▼
WebApplication.CreateBuilder(args)
           |
           ├── builder.Configuration (appsettings.json, Environment Variables, User Secrets)
           |
           ├── builder.Services.Add...() (Register DbContext, JWT, CORS, DI Services)
           |
           ▼
builder.Build() ──────────────────► Transition from Service Configuration to HTTP Pipeline
           |
           ├── app.UseMiddleware<...>() (Global Exception Handling)
           ├── app.UseCors(...)
           ├── app.UseAuthentication()
           ├── app.UseAuthorization()
           ├── app.MapControllers()
```

```
           |
           ▼
app.Run() ──────────────────────► Starts Kestrel Web Server listening for HTTP requests
```

Complete Annotated `Program.cs` Implementation

```
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using SkillHub.API.Middlewares;
using SkillHub.Application.Features.Courses.Interfaces;
using SkillHub.Application.Features.Identity.Interfaces;
using SkillHub.Application.Features.Support.Interfaces;
using SkillHub.Infrastructure.Authentication;
using SkillHub.Infrastructure.Persistence.Context;
using SkillHub.Infrastructure.Services.Courses;
using SkillHub.Infrastructure.Services.Support;

// 1. Initialize the WebApplicationBuilder
var builder = WebApplication.CreateBuilder(args);

// ============================================================================
// 2. CONFIGURE SERVICES (builder.Services) → Inversion of Control Container
// ============================================================================

// A. Register Controllers & API Explorers
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();

// B. Register EF Core ApplicationDbContext with SQL Server
builder.Services.AddDbContext<ApplicationDbContext>(options =>
{
    var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
    options.UseSqlServer(connectionString, sqlOptions =>
    {
        sqlOptions.EnableRetryOnFailure(
            maxRetryCount: 5,
            maxRetryDelay: TimeSpan.FromSeconds(30),
            errorNumbersToAdd: null);
    });
});

// C. Register AutoMapper for DTO Object Projections
builder.Services.AddAutoMapper(AppDomain.CurrentDomain.GetAssemblies());

// D. Configure Authentication & JWT Bearer Token Validation
var jwtSecret = builder.Configuration["Jwt:Key"]
    ?? throw new InvalidOperationException("JWT Secret Key is missing from configuration.");
var jwtIssuer = builder.Configuration["Jwt:Issuer"] ?? "SkillHubAPI";
var jwtAudience = builder.Configuration["Jwt:Audience"] ?? "SkillHubClient";

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.RequireHttpsMetadata = false; // Set to true in strict production SSL
    options.SaveToken = true;
    options.TokenValidationParameters = new TokenValidationParameters
```

```
    {
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSecret)),
        ValidateIssuer = true,
        ValidIssuer = jwtIssuer,
        ValidateAudience = true,
        ValidAudience = jwtAudience,
        ValidateLifetime = true,
        ClockSkew = TimeSpan.Zero // Eliminates the default 5-minute grace period
    };
});

// E. Configure Role & Policy Authorization
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireSuperAdmin", policy => policy.RequireRole("SuperAdmin"));
    options.AddPolicy("RequireInstructor", policy => policy.RequireRole("Instructor"));
});

// F. Configure Cross-Origin Resource Sharing (CORS)
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAngularClient", policy =>
    {
        policy.WithOrigins("http://localhost:4200", "https://skillhub.azurewebsites.net")
              .AllowAnyHeader()
              .AllowAnyMethod()
              .AllowCredentials();
    });
});

// G. Register Application Business Services with Appropriate Lifetimes
builder.Services.AddScoped<ICourseService, CourseService>();
builder.Services.AddScoped<ISupportService, SupportService>();
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddSingleton<IJwtTokenGenerator, JwtTokenGenerator>();

// H. Configure Swagger / OpenAPI Documentation with JWT Support
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "SkillHub Enterprise API", Version = "v1" });
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Description = "JWT Authorization header using the Bearer scheme. Enter 'Bearer' [space] and your
token.",
        Name = "Authorization",
        In = ParameterLocation.Header,
        Type = SecuritySchemeType.ApiKey,
        Scheme = "Bearer"
    });
    c.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference { Type = ReferenceType.SecurityScheme, Id = "Bearer" }
            },
            Array.Empty<string>()
        }
    });
});

// ==========================================================================
// 3. BUILD APPLICATION → Transition to HTTP Request Pipeline
// ==========================================================================
```

```
var app = builder.Build();

// ============================================================================
// 4. CONFIGURE HTTP REQUEST PIPELINE (Middleware Chain)
// ============================================================================

// Global Exception Handler must be the very first middleware to catch all unhandled errors
app.UseMiddleware<GlobalExceptionMiddleware>();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(c => c.SwaggerEndpoint("/swagger/v1/swagger.json", "SkillHub API v1"));
}

app.UseHttpsRedirection();
app.UseStaticFiles(); // Serves course thumbnails, profile avatars, PDFs

app.UseRouting();

// CORS must execute after UseRouting and before UseAuthentication
app.UseCors("AllowAngularClient");

// Authentication verifies WHO the user is (validates JWT)
app.UseAuthentication();

// Authorization verifies WHAT the user is allowed to do (checks Roles/Permissions)
app.UseAuthorization();

// Map route endpoints to controller actions
app.MapControllers();

app.Run();
```

---

3. Dependency Injection (DI) & Inversion of Control (IoC)

What is Inversion of Control (IoC)?

In traditional programming, high-level code instantiates its own dependencies directly:

```
// Anti-pattern: High-level class tightly coupled to concrete implementation
public class CourseController : ControllerBase
{
    private readonly CourseService _courseService = new CourseService(new ApplicationDbContext());
}
```

Inversion of Control flips this responsibility: objects do not create their dependencies; instead, an external framework (the **IoC Container**) creates and injects dependencies when needed.

What is Dependency Injection (DI)?

Dependency Injection is the concrete software design pattern used to achieve Inversion of Control. In ASP.NET Core, dependencies are injected via **Constructor Injection**.

```
┌──────────────────────────────────┐
│     ASP.NET Core IoC Container   │
│  (ServiceCollection / ServiceProvider)│
└──────────────────┬───────────────┘
```



The Three Service Lifetimes

Choosing the correct lifetime is critical for application reliability, concurrency safety, and memory management:

LIFETIME	METHOD	LIFECYCLE DESCRIPTION	BEST USE CASES	PITFALL / RISK
Transient	`AddTransient<TInterface, TImplementation>()`	Created **every single time** it is requested. If 3 classes in the same HTTP request ask for it, 3 separate instances are created.	Lightweight, stateless utilities, formatters, calculators, validators.	Heavy initialization overhead if misused on large objects.
Scoped	`AddScoped<TInterface, TImplementation>()`	Created **once per HTTP request**. All classes handling the same HTTP request share the identical instance. Destroyed when request ends.	`DbContext`, Repositories, Domain Services, Unit of Work, Current User Context.	Thread-unsafe if shared across asynchronous background worker threads.
Singleton	`AddSingleton<TInterface, TImplementation>()`	Created **once when application starts** and lives for the entire process lifetime. All users and requests share this single instance.	In-memory caches, JWT token generators, background queue workers, configuration wrappers.	Memory leaks if holding references; **must be strictly thread-safe**.

> ⚠️ **WARNING**
>
> > **The Captive Dependency Pitfall** > Never inject a `Scoped` service (such as `ApplicationDbContext`) into a `Singleton` service. Doing so causes the scoped instance to be kept alive forever by the singleton, leading to stale database tracking data, concurrency exceptions on simultaneous requests, and memory exhaustion.

Complete Dependency Injection Code Example

1. Define the Interface Contract (`SkillHub.Application`)

```
namespace SkillHub.Application.Features.Courses.Interfaces;

public interface ICourseService
{
    Task<List<CourseListDto>> GetAllPublishedCoursesAsync();
    Task<CourseResponseDto?> GetByIdAsync(long id);
    Task<CourseResponseDto> CreateAsync(CreateCourseRequestDto request, long instructorId);
    Task DeleteAsync(long id, long instructorId);
}
```

2. Implement the Service Concrete Class (`SkillHub.Infrastructure`)

```
namespace SkillHub.Infrastructure.Services.Courses;

public class CourseService : ICourseService
{
    private readonly ApplicationDbContext _context;
    private readonly IMapper _mapper;
    private readonly ILogger<CourseService> _logger;

    // All dependencies are injected through the constructor by the IoC Container
    public CourseService(
        ApplicationDbContext context,
        IMapper mapper,
        ILogger<CourseService> logger)
    {
        _context = context ?? throw new ArgumentNullException(nameof(context));
        _mapper = mapper ?? throw new ArgumentNullException(nameof(mapper));
        _logger = logger ?? throw new ArgumentNullException(nameof(logger));
    }

    public async Task<List<CourseListDto>> GetAllPublishedCoursesAsync()
    {
        _logger.LogInformation("Retrieving all published courses from database.");

        return await _context.Courses
            .AsNoTracking()
            .Where(c => c.IsPublished)
            .OrderByDescending(c => c.CreatedAt)
            .ProjectTo<CourseListDto>(_mapper.ConfigurationProvider)
            .ToListAsync();
    }

    public async Task<CourseResponseDto?> GetByIdAsync(long id)
    {
        return await _context.Courses
            .AsNoTracking()
            .Include(c => c.Category)
            .Include(c => c.Instructor)
            .Where(c => c.Id == id)
            .ProjectTo<CourseResponseDto>(_mapper.ConfigurationProvider)
            .FirstOrDefaultAsync();
    }

    public async Task<CourseResponseDto> CreateAsync(CreateCourseRequestDto request, long instructorId)
    {
        var course = _mapper.Map<Course>(request);
        course.InstructorId = instructorId;
        course.CreatedAt = DateTime.UtcNow;

        await _context.Courses.AddAsync(course);
        await _context.SaveChangesAsync();

        _logger.LogInformation("Course {CourseId} successfully created by Instructor {InstructorId}.",
course.Id, instructorId);
        return _mapper.Map<CourseResponseDto>(course);
    }

    public async Task DeleteAsync(long id, long instructorId)
    {
        var course = await _context.Courses.FindAsync(id);
        if (course == null)
            throw new KeyNotFoundException($"Course with ID {id} not found.");
```

```
        if (course.InstructorId != instructorId)
            throw new UnauthorizedAccessException("You are not authorized to delete this course.");

        _context.Courses.Remove(course);
        await _context.SaveChangesAsync();
    }
}
```

3. Inject and Consume in the Controller (`SkillHub.API`)

```
namespace SkillHub.API.Controllers;

[ApiController]
[Route("api/[controller]")]
public class CourseController : ControllerBase
{
    private readonly ICourseService _courseService;

    // Controller receives ICourseService without knowing anything about ApplicationDbContext
    public CourseController(ICourseService courseService)
    {
        _courseService = courseService;
    }

    [HttpGet]
    [AllowAnonymous]
    public async Task<ActionResult<List<CourseListDto>>> GetAll()
    {
        var courses = await _courseService.GetAllPublishedCoursesAsync();
        return Ok(courses);
    }
}
```

---

4. Entity Framework Core 8 (EF Core) Deep Dive

Entity Framework Core is Microsoft's high-performance **Object-Relational Mapper (ORM)**. It translates C# LINQ queries directly into optimized SQL queries, handles schema migrations, manages database transactions, and coordinates database writes.

Domain Entities & Relationship Configurations

Entity Class (`SkillHub.Domain`)

```
namespace SkillHub.Domain.Entities.Courses;

public class Course
{
    public long Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public bool IsPublished { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    // Foreign Keys & Navigation Properties
    public long InstructorId { get; set; }
```

```
    public User Instructor { get; set; } = null!;

    public int? CategoryId { get; set; }
    public Category? Category { get; set; }

    public ICollection<CourseSection> Sections { get; set; } = new List<CourseSection>();
    public ICollection<Enrollment> Enrollments { get; set; } = new List<Enrollment>();
}
```

ApplicationDbContext & Fluent API Configuration (`SkillHub.Infrastructure`)

Fluent API configuration via `OnModelCreating` is superior to Data Annotations because it maintains complete domain purity:

```
namespace SkillHub.Infrastructure.Persistence.Context;

public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }

    public DbSet<User> Users => Set<User>();
    public DbSet<Role> Roles => Set<Role>();
    public DbSet<UserRole> UserRoles => Set<UserRole>();
    public DbSet<Course> Courses => Set<Course>();
    public DbSet<Enrollment> Enrollments => Set<Enrollment>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // 1. Composite Primary Key for Many-to-Many Join Table
        modelBuilder.Entity<UserRole>(entity =>
        {
            entity.HasKey(ur => new { ur.UserId, ur.RoleId });

            entity.HasOne(ur => ur.User)
                  .WithMany(u => u.UserRoles)
                  .HasForeignKey(ur => ur.UserId)
                  .OnDelete(DeleteBehavior.Cascade);

            entity.HasOne(ur => ur.Role)
                  .WithMany(r => r.UserRoles)
                  .HasForeignKey(ur => ur.RoleId)
                  .OnDelete(DeleteBehavior.Cascade);
        });

        // 2. Course Entity Configuration & Constraints
        modelBuilder.Entity<Course>(entity =>
        {
            entity.HasKey(c => c.Id);
            entity.Property(c => c.Title).IsRequired().HasMaxLength(200);
            entity.Property(c => c.Price).HasPrecision(18, 2);

            // One Instructor (User) has Many Courses
            entity.HasOne(c => c.Instructor)
                  .WithMany(u => u.Courses)
                  .HasForeignKey(c => c.InstructorId)
                  .OnDelete(DeleteBehavior.Restrict); // Prevent deleting instructor if courses exist

            // Indexes for fast searching
            entity.HasIndex(c => c.IsPublished);
            entity.HasIndex(c => c.Title);
```

```
        });
    }
}
```

Essential EF Core Concepts

1. Change Tracking vs `AsNoTracking()`

- **Tracked Queries (Default)**: EF Core stores a snapshot of retrieved entities in its internal Change Tracker. When `SaveChangesAsync()` is called, EF compares entities against the snapshot and generates SQL `UPDATE` statements for altered columns.
- **Untracked Queries (`.AsNoTracking()`)**: EF Core bypasses the change tracker entirely. Use this for **all read-only queries**. It eliminates snapshot memory overhead and yields a **30%–50% performance increase**.

```
// Read-only query: FAST, low memory
var courses = await _context.Courses.AsNoTracking().Where(c => c.IsPublished).ToListAsync();

// Update query: MUST be tracked so EF knows what changed
var course = await _context.Courses.FirstOrDefaultAsync(c => c.Id == id);
course.Price = 49.99m;
await _context.SaveChangesAsync(); // Generates UPDATE Courses SET Price = 49.99 WHERE Id = @id
```

2. Eager Loading vs Projection

- **Eager Loading (`Include` / `ThenInclude`)**: Loads related entities via SQL `JOIN`.
- **Projection (`Select` / `ProjectTo`)**: Retrieves **only the exact columns needed**, ignoring unused database columns.

```
// Eager Loading (Loads ALL columns of Course, Category, and Instructor)
var course = await _context.Courses
    .Include(c => c.Category)
    .Include(c => c.Instructor)
    .FirstOrDefaultAsync(c => c.Id == id);

// SQL Projection (Generates SELECT c.Id, c.Title, u.FirstName FROM Courses c JOIN Users u...)
var dto = await _context.Courses
    .Where(c => c.Id == id)
    .Select(c => new CourseListDto
    {
        Id = c.Id,
        Title = c.Title,
        InstructorName = c.Instructor.FirstName + " " + c.Instructor.LastName
    })
    .FirstOrDefaultAsync();
```

---

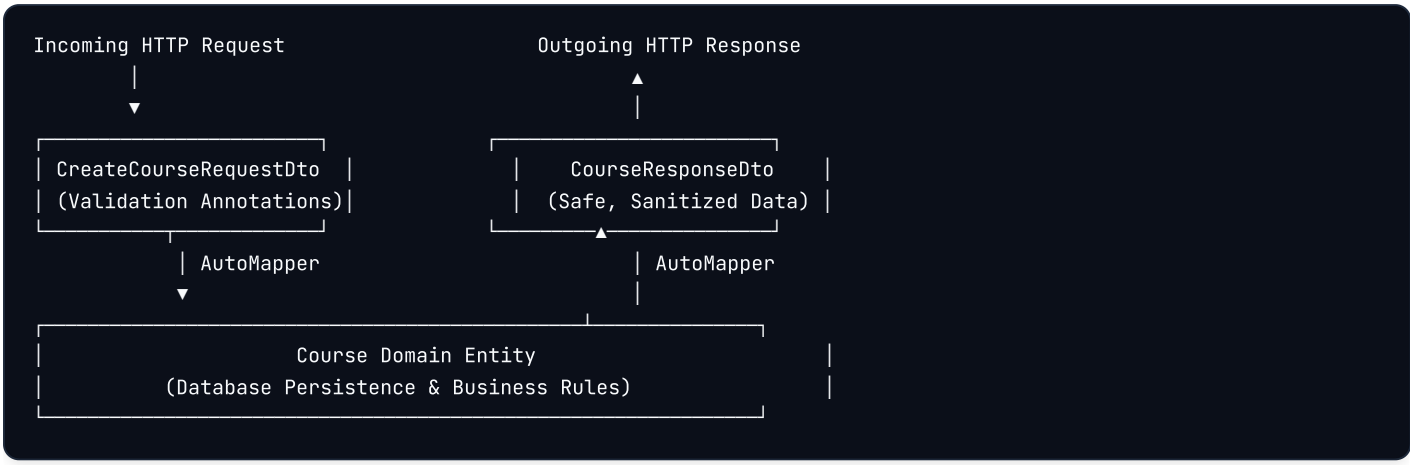
5. Data Transfer Objects (DTOs) & Object Mapping

Why Are DTOs Mandatory?

Exposing raw EF Core Domain Entities directly through API controller endpoints introduces critical vulnerabilities:

1. **Over-Posting Attacks (Mass Assignment)**: A malicious user can send `{"Id": 5, "IsAdmin": true, "PasswordHash": "..."}` in a JSON payload. If the controller binds directly to the `User` entity, EF Core will overwrite administrative fields.

2. **Circular Serialization References**: If `Course` has a list of `Enrollments`, and `Enrollment` has a reference back to `Course`, `System.Text.Json` throws a fatal `JsonException: A possible object cycle was detected`.
3. **Payload Bloat & Data Hiding**: Domain models contain audit properties (`PasswordHash`, `SecurityStamp`, `ConcurrencyStamp`, `RowVersion`). DTOs ensure sensitive properties never leave the server boundary.



Complete DTO & AutoMapper Implementation

1. Request DTO with Validation Data Annotations

```
namespace SkillHub.Application.Features.Courses.DTOs;

public class CreateCourseRequestDto
{
    [Required(ErrorMessage = "Course title is mandatory.")]
    [StringLength(150, MinimumLength = 5, ErrorMessage = "Title must be between 5 and 150 characters.")]
    public string Title { get; set; } = string.Empty;

    [Required(ErrorMessage = "Course description cannot be blank.")]
    [MaxLength(2000, ErrorMessage = "Description cannot exceed 2000 characters.")]
    public string Description { get; set; } = string.Empty;

    [Range(0.00, 9999.99, ErrorMessage = "Price must be between $0.00 and $9,999.99.")]
    public decimal Price { get; set; }

    [Required]
    public int CategoryId { get; set; }
}
```

2. Response DTO

```
namespace SkillHub.Application.Features.Courses.DTOs;

public class CourseResponseDto
{
    public long Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public bool IsPublished { get; set; }
    public string CategoryName { get; set; } = string.Empty;
    public string InstructorName { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }
}
```

3. AutoMapper Profile Definition

```
namespace SkillHub.Application.Common.Mappings;

public class MappingProfile : Profile
{
    public MappingProfile()
    {
        // Entity to DTO mapping with custom property resolution
        CreateMap<Course, CourseResponseDto>()
            .ForMember(dest => dest.CategoryName, opt => opt.MapFrom(src => src.Category != null ?
src.Category.Name : "Uncategorized"))
            .ForMember(dest => dest.InstructorName, opt => opt.MapFrom(src => src.Instructor != null ? $"
{src.Instructor.FirstName} {src.Instructor.LastName}" : "Unknown"));

        // DTO to Entity mapping
        CreateMap<CreateCourseRequestDto, Course>()
            .ForMember(dest => dest.Id, opt => opt.Ignore())
            .ForMember(dest => dest.CreatedAt, opt => opt.Ignore())
            .ForMember(dest => dest.IsPublished, opt => opt.MapFrom(_ => false));
    }
}
```

---

6. Model Binding & HTTP Parameter Resolution

What is Model Binding?

Model Binding is the ASP.NET Core subsystem that extracts raw data from various parts of an incoming HTTP request (Route tokens, Query strings, Request Headers, JSON bodies, Multi-part form data) and converts them into strongly-typed C# parameters and objects.

Model Binding Sources Comparison

ATTRIBUTE	SOURCE IN HTTP REQUEST	EXAMPLE URI OR PAYLOAD	IDEAL USE CASE
`[FromBody]`	Request Body (JSON/XML)	`{"title": "C# Masterclass", "price": 49.99}`	Complex object creation and updates (`POST` , `PUT` , `PATCH`).
`[FromRoute]`	URL Path Segment	`/api/courses/42`	Unique resource identifiers (`id:long` , `slug:string`).
`[FromQuery]`	URL Query String	`/api/courses?page=2&pageSize=10&search=net`	Pagination, filtering, searching, and sorting parameters.
`[FromHeader]`	HTTP Request Headers	`Authorization: Bearer <token>` , `X-Api-Version: 2`	Metadata, API keys, tenant identifiers, localization headers.
`[FromForm]`	`multipart/form-data`	Multi-part boundary with files + form fields	File uploads (profile photos, video lessons, course thumbnails).

Complete Controller Demonstrating All Model Binding Sources

```
namespace SkillHub.API.Controllers;

[ApiController]
[Route("api/[controller]")]
public class CoursesController : ControllerBase
{
    // 1. [FromRoute] and [FromQuery] Combination
    // GET /api/courses/category/5?page=1&pageSize=20&search=react
    [HttpGet("category/{categoryId:int}")]
    public async Task<IActionResult> GetCoursesByCategory(
        [FromRoute] int categoryId,
        [FromQuery] int page = 1,
        [FromQuery] int pageSize = 10,
        [FromQuery] string? search = null)
    {
        return Ok(new { categoryId, page, pageSize, search });
    }

    // 2. [FromBody] Binding for Complex JSON DTOs
    // POST /api/courses
    [HttpPost]
    [Authorize(Roles = "Instructor")]
    public async Task<IActionResult> CreateCourse([FromBody] CreateCourseRequestDto request)
    {
        // [ApiController] automatically validates ModelState and returns 400 BadRequest if invalid
        var instructorId = long.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier)!);
        // ... invoke service
        return CreatedAtAction(nameof(GetById), new { id = 101 }, request);
    }

    // 3. [FromRoute] Binding
    // GET /api/courses/101
    [HttpGet("{id:long}")]
    public async Task<IActionResult> GetById([FromRoute] long id)
    {
        return Ok(new { id, title = "ASP.NET Core Architecture" });
    }

    // 4. [FromForm] Binding for File Uploads
    // POST /api/courses/101/thumbnail
    [HttpPost("{id:long}/thumbnail")]
    [Authorize(Roles = "Instructor")]
    public async Task<IActionResult> UploadThumbnail(
        [FromRoute] long id,
        [FromForm] IFormFile file)
    {
        if (file == null || file.Length == 0)
            return BadRequest("No file was uploaded.");

        var allowedExtensions = new[] { ".jpg", ".jpeg", ".png" };
        var extension = Path.GetExtension(file.FileName).ToLowerInvariant();
        if (!allowedExtensions.Contains(extension))
            return BadRequest("Only JPG and PNG images are allowed.");

        return Ok(new { id, fileName = file.FileName, sizeBytes = file.Length });
    }

    // 5. [FromHeader] Binding
    // GET /api/courses/client-audit
    [HttpGet("client-audit")]
    public IActionResult AuditClient([FromHeader(Name = "User-Agent")] string userAgent)
    {
```

```
        return Ok(new { clientBrowser = userAgent });
    }
}
```

7. Controllers & RESTful API Orchestration

`ControllerBase` vs `Controller`

- `ControllerBase`: Designed exclusively for Web APIs. Provides access to `HttpContext`, `User`, `Request`, `Response`, and standard action result helpers (`Ok()`, `BadRequest()`, `NotFound()`, `Unauthorized()`).
- `Controller`: Inherits from `ControllerBase` and adds MVC Razor View rendering support (`View()`, `ViewData`, `ViewBag`). Always inherit from `ControllerBase` in pure Web APIs to avoid unnecessary overhead.

The Role of `[ApiController]`

Decorating an API controller with `[ApiController]` unlocks automatic developer-friendly behaviors: 1. **Automatic Model Validation**: If `ModelState.IsValid == false`, ASP.NET Core automatically short-circuits the pipeline and returns `400 Bad Request` containing an RFC 7807 `ProblemDetails` response. You never need to write `if (!ModelState.IsValid) return BadRequest();`. 2. **Inferred Binding Sources**: Automatically assumes complex types come `[FromBody]` and primitive types come `[FromRoute]` or `[FromQuery]`. 3. **Multipart/Form-Data Inference**: Parameters of type `IFormFile` or `IFormFileCollection` automatically bind to `[FromForm]`.

Standard HTTP Status Codes & Action Results

HTTP VERB	INTENT	SUCCESS CODE	METHOD HELPER	ERROR SCENARIOS
`GET`	Retrieve resource	`200 OK`	`Ok(data)`	`404 NotFound()`
`POST`	Create new resource	`201 Created`	`CreatedAtAction(nameof(...), routeValues, data)`	`400 BadRequest()`, `409 Conflict()`
`PUT`	Full replacement of resource	`200 OK` or `204 NoContent`	`Ok()` or `NoContent()`	`400 BadRequest()`, `404 NotFound()`
`PATCH`	Partial update of resource	`200 OK`	`Ok(updatedData)`	`400 BadRequest()`
`DELETE`	Remove resource	`204 NoContent`	`NoContent()`	`404 NotFound()`, `403 Forbid()`

Production Controller Architecture

```
namespace SkillHub.API.Controllers;

[ApiController]
[Route("api/[controller]")]
[Produces("application/json")]
public class SupportController : ControllerBase
{
    private readonly ISupportService _supportService;

    public SupportController(ISupportService supportService)
```

```
    {
        _supportService = supportService;
    }

    [HttpGet("admin/tickets")]
    [Authorize(Roles = "SuperAdmin,SubAdmin,ProductSupport")]
    [ProducesResponseType(StatusCodes.Status200OK, Type = typeof(List<TicketSummaryDto>))]
    [ProducesResponseType(StatusCodes.Status401Unauthorized)]
    [ProducesResponseType(StatusCodes.Status403Forbidden)]
    public async Task<IActionResult> GetAllTickets([FromQuery] string? status, [FromQuery] long? assignedToId)
    {
        var tickets = await _supportService.GetAdminTicketsAsync(status, assignedToId);
        return Ok(tickets);
    }

    [HttpPut("admin/tickets/{id:long}/status")]
    [Authorize(Roles = "ProductSupport,SubAdmin")]
    [ProducesResponseType(StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status403Forbidden)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> UpdateTicketStatus(
        [FromRoute] long id,
        [FromBody] UpdateTicketStatusRequestDto request)
    {
        var adminId = long.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier)!);
        var isSubAdmin = User.IsInRole("SubAdmin");

        // SubAdmins are constrained to updating only tickets assigned to them
        if (isSubAdmin)
        {
            var ticket = await _supportService.GetTicketDetailsAsync(id);
            if (ticket == null) return NotFound("Ticket not found.");
            if (ticket.AssignedAdminId ≠ adminId)
            {
                return StatusCode(StatusCodes.Status403Forbidden,
                    "SubAdmins may only update the status of tickets assigned to them.");
            }
        }

        await _supportService.UpdateStatusAsync(id, request.Status, request.Message, adminId);
        return Ok(new { message = $"Ticket status updated to {request.Status} successfully." });
    }
}
```

8. Role-Based Authentication (RBAC) & Claims Authorization

Authentication vs Authorization

- **Authentication ("AuthN")**: *Who are you?* Validating credentials (email/password) and producing a cryptographically verified identity (JWT Bearer Token). Failure results in `401 Unauthorized`.
- **Authorization ("AuthZ")**: *What are you allowed to do?* Checking if the authenticated identity possesses the necessary **Role** (`SuperAdmin`, `SubAdmin`, `Instructor`, `Student`, `ProductSupport`) or **Permission** (`Tabs.Users.View`). Failure results in `403 Forbidden`.

Anatomy of a JSON Web Token (JWT)

A JWT consists of three Base64Url-encoded segments separated by dots (`.`): `header.payload.signature`.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI0MiIsIm5hbWVpZCI6IjQyIiwiZW1haWwiOiJhbGV4QHNoaWxsLmNvbSIsInJvbGUiOiJJbnN0cnVjdG9yIiwianRpIjoiYWJjLTEyMyIsImV4cCI6MTgwMDAwMDAwMH0.K2zP8x...
  ▲                                    ▲                                                                    ▲
  |                                    |                                                                    |
1. HEADER                            2. PAYLOAD (Claims)                                                  3.
SIGNATURE
   Algorithm: HMAC-SHA256               sub: 42 (User Id)                                                      HMAC-
SHA256(
   Type: JWT                            email: alex@skill.com
base64Url(Header) + "." +
                                        role: Instructor
base64Url(Payload),
                                        exp: Expiration Timestamp
SecretKey
                                                                                                             )
```

Complete JWT Token Generator (`SkillHub.Infrastructure`)

```
namespace SkillHub.Infrastructure.Authentication;

public class JwtTokenGenerator : IJwtTokenGenerator
{
    private readonly IConfiguration _configuration;

    public JwtTokenGenerator(IConfiguration configuration)
    {
        _configuration = configuration;
    }

    public string GenerateToken(User user)
    {
        var secretKey = _configuration["Jwt:Key"]
            ?? throw new InvalidOperationException("Jwt:Key is not configured.");
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey));
        var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        // Claims represent statements about the user entity
        var claims = new List<Claim>
        {
            new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
            new Claim("nameid", user.Id.ToString()), // For compatibility across client frameworks
            new Claim(JwtRegisteredClaimNames.Email, user.Email),
            new Claim(ClaimTypes.Email, user.Email),
            new Claim("fullName", $"{user.FirstName} {user.LastName}"),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
        };

        // Add Role Claims
        foreach (var userRole in user.UserRoles)
        {
            claims.Add(new Claim(ClaimTypes.Role, userRole.Role.Name));
            claims.Add(new Claim("role", userRole.Role.Name));
        }

        // Add Fine-Grained Permission Claims
        foreach (var userPermission in user.UserPermissions.Where(up => up.IsGranted))
        {
            claims.Add(new Claim("permission", userPermission.Permission.Name));
        }

        var tokenDescriptor = new SecurityTokenDescriptor
```

```
        {
            Subject = new ClaimsIdentity(claims),
            Expires = DateTime.UtcNow.AddHours(2), // Access token expiration
            Issuer = _configuration["Jwt:Issuer"] ?? "SkillHubAPI",
            Audience = _configuration["Jwt:Audience"] ?? "SkillHubClient",
            SigningCredentials = credentials
        };

        var tokenHandler = new JwtSecurityTokenHandler();
        var token = tokenHandler.CreateToken(tokenDescriptor);
        return tokenHandler.WriteToken(token);
    }
}
```

Refresh Token Rotation Pattern

Access tokens must be short-lived (e.g., 15 minutes to 2 hours) to limit damage if intercepted. Long-lived sessions are maintained using **Refresh Token Rotation**: 1. When an access token expires, the client sends `POST /api/auth/refresh` containing the expired JWT and the opaque refresh token. 2. The server validates that the refresh token exists in the database, is not revoked, and is not expired. 3. The server immediately **revokes the used refresh token**, issues a brand-new access token + refresh token pair, and persists the new token in the database. 4. If a compromised refresh token is reused, the server detects token reuse and invalidates all active sessions for that user account.

---

9. HTTP Middleware Pipeline & Error Handling

The Russian-Doll Execution Model

ASP.NET Core processes requests using a sequential chain of delegate components called **Middleware**. Each middleware can: 1. Execute logic **before** passing the request to the next component via `await _next(context)`. 2. Short-circuit the pipeline by deciding not to call `_next(context)` (e.g., authentication failure). 3. Execute logic **after** the downstream component completes on the outbound response journey.



Middleware Ordering Rules

Middleware order in `Program.cs` is strictly critical: 1. `UseExceptionHandler` or `UseMiddleware<GlobalExceptionMiddleware>()` must be **first** so it wraps the entire pipeline. 2. `UseRouting()` matches incoming URL to an endpoint. 3. `UseCors()` must be placed **after** `UseRouting()` and **before** `UseAuthentication()`. 4. `UseAuthentication()` must precede `UseAuthorization()` — you cannot evaluate roles on an identity that has not been authenticated yet. 5. `MapControllers()` must be **last** to execute the resolved controller action.

Complete Production Global Exception Middleware

```
namespace SkillHub.API.Middlewares;

public class GlobalExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<GlobalExceptionMiddleware> _logger;
    private readonly IHostEnvironment _env;

    public GlobalExceptionMiddleware(
        RequestDelegate next,
        ILogger<GlobalExceptionMiddleware> logger,
        IHostEnvironment env)
    {
        _next = next;
        _logger = logger;
        _env = env;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            // Proceed down the pipeline
            await _next(context);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "An unhandled exception occurred while processing request {Path}",
context.Request.Path);
            await HandleExceptionAsync(context, ex);
        }
    }

    private Task HandleExceptionAsync(HttpContext context, Exception exception)
    {
        context.Response.ContentType = "application/json";

        var statusCode = exception switch
        {
            KeyNotFoundException => HttpStatusCode.NotFound,
            UnauthorizedAccessException => HttpStatusCode.Forbidden,
            ArgumentException or InvalidOperationException => HttpStatusCode.BadRequest,
            _ => HttpStatusCode.InternalServerError
        };

        context.Response.StatusCode = (int)statusCode;

        var response = new
        {
            status = context.Response.StatusCode,
            title = GetTitleForStatusCode(statusCode),
            message = exception.Message,
```

```
            // Include stack trace only in local development; never expose internals in production
            detail = _env.IsDevelopment() ? exception.StackTrace : null,
            timestamp = DateTime.UtcNow
        };

        var json = JsonSerializer.Serialize(response, new JsonSerializerOptions
        {
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase
        });

        return context.Response.WriteAsync(json);
    }

    private static string GetTitleForStatusCode(HttpStatusCode statusCode) => statusCode switch
    {
        HttpStatusCode.NotFound => "Resource Not Found",
        HttpStatusCode.Forbidden => "Access Forbidden",
        HttpStatusCode.BadRequest => "Invalid Request",
        _ => "Internal Server Error"
    };
}
```

10. End-to-End Request Flow: Tracing a Request Lifecycle

To understand how all these systems collaborate in production, let us trace an authenticated instructor creating a new course:

```
[1. Client Browser (Angular)]
     | HTTP POST https://api.skillhub.com/api/course
     | Headers: Authorization: Bearer eyJhbGci...
     | Body: {"title": "Fullstack .NET 8", "price": 49.99, "categoryId": 3}
     ▼
[2. Kestrel Web Server & Middleware Pipeline]
     | GlobalExceptionMiddleware: Starts try/catch block
     | CORS Middleware: Verifies Origin header matches 'http://localhost:4200'
     | Authentication Middleware: Decodes JWT, validates HMAC-SHA256 signature,
     |   verifies expiry date, builds ClaimsPrincipal (User.Claims)
     | Authorization Middleware: Evaluates [Authorize(Roles = "Instructor")]. User has Role claim → Passes
     ▼
[3. Routing & Model Binding Engine]
     | Matches route template "api/[controller]" → CourseController.Create
     | Model Binder reads request stream, deserializes JSON into CreateCourseRequestDto
     | Runs DataAnnotations validation on DTO. All fields valid → ModelState.IsValid = true
     ▼
[4. Controller Layer (CourseController)]
     | Controller constructor receives ICourseService via Constructor Dependency Injection
     | Extracts instructorId from User.FindFirst(ClaimTypes.NameIdentifier)
     | Calls await _courseService.CreateAsync(request, instructorId)
     ▼
[5. Service Layer (CourseService)]
     | AutoMapper maps CreateCourseRequestDto → Course Domain Entity
     | Sets course.InstructorId = instructorId, course.CreatedAt = DateTime.UtcNow
     | Calls await _context.Courses.AddAsync(course)
     ▼
[6. Persistence Layer (EF Core & SQL Server)]
     | EF Core Change Tracker marks Course entity state as 'Added'
     | await _context.SaveChangesAsync() executes
     | EF Core generates parameterized SQL INSERT statement:
     |   INSERT INTO Courses (Title, Price, CategoryId, InstructorId, CreatedAt)
     |   VALUES (@p0, @p1, @p2, @p3, @p4); SELECT SCOPE_IDENTITY();
```

```
      | SQL Server executes query and returns generated primary key (Id = 101)
      ▼
[7. Return Journey & Outbound Response]
      | Service maps Course entity → CourseResponseDto
      | Controller returns CreatedAtAction(nameof(GetById), new { id = 101 }, responseDto)
      | Response passes back through middleware chain (HTTP 201 Created)
      ▼
[8. Angular UI Client]
      | Receives HTTP 201 Created with JSON body
      | Navigates instructor to curriculum builder for Course ID 101
```

---

11. Technical Interview & Viva Voce Master Cheat Sheet

1. What is the difference between `AddTransient`, `AddScoped`, and `AddSingleton`?

- **Answer**:
- `AddTransient` creates a new instance **every time** it is requested. Best for lightweight, stateless services.
- `AddScoped` creates one instance **per HTTP request**. Best for database contexts (`DbContext`) and business services.
- `AddSingleton` creates a single instance **once when the application starts** and reuses it across all requests. Best for caching and token generation.

2. Why should you never inject a Scoped service into a Singleton service?

- **Answer**: This is called the **Captive Dependency** anti-pattern. The Singleton service will hold onto the Scoped instance for the entire lifetime of the application, turning the scoped service into an unintended singleton. In the case of `DbContext`, this causes memory leaks, concurrency violations, and stale tracked data.

3. What is the difference between `IEnumerable<T>`, `IQueryable<T>`, and `List<T>` in EF Core?

- **Answer**:
- `IQueryable<T>` represents a query definition that has **not yet executed**. Filtering operations (`Where`, `Take`) are translated into SQL and executed on the database server.
- `IEnumerable<T>` operates in server memory. If you filter an `IEnumerable`, the entire table has already been loaded into memory.
- `List<T>` is the fully materialized in-memory result of an executed query.

4. What is `.AsNoTracking()` in EF Core and when should it be used?

- **Answer**: It instructs EF Core's Change Tracker not to maintain identity or snapshot information for query results. It should be used on all **read-only queries** to reduce memory consumption and improve execution speed by 30%–50%.

5. Why are DTOs mandatory instead of exposing Domain Entities directly?

- **Answer**:
- Prevents **Over-Posting (Mass Assignment)** attacks.
- Eliminates **circular serialization references** during JSON output.
- Hides sensitive domain data (e.g., `PasswordHash`).
- Optimizes network bandwidth by sending only required fields.

6. What does the `[ApiController]` attribute do in ASP.NET Core?

- **Answer**: It enables opinionated API behaviors:

- Automatically validates model state and returns `400 Bad Request` with `ProblemDetails` if validation fails.
- Infers parameter binding sources (`[FromBody]` for complex types, `[FromRoute]` for route parameters).
- Handles multipart/form-data for `IFormFile` parameters.

7. What is the difference between Authentication and Authorization?

- **Answer**:
- **Authentication** verifies the user's identity (*Who are you?*). Returns `401 Unauthorized` on failure.
- **Authorization** verifies the user's permissions and roles (*What are you allowed to do?*). Returns `403 Forbidden` on failure.

8. What is the difference between `[Authorize(Roles = "Admin,Instructor")]` and `[Authorize(Roles = "Admin")] [Authorize(Roles = "Instructor")]`?

- **Answer**:
- Comma-separated (`Roles = "Admin,Instructor"`) evaluates as an **OR** condition: the user must belong to either Admin *OR* Instructor.
- Multiple distinct attributes evaluate as an **AND** condition: the user must possess *BOTH* the Admin and Instructor roles.

9. Why is the order of middleware in `Program.cs` critical?

- **Answer**: Middleware executes in the exact order it is declared. If `UseAuthorization()` is placed before `UseAuthentication()`, authorization checks will execute against an unauthenticated identity (`ClaimsPrincipal` is empty), blocking all protected endpoints. Similarly, exception-handling middleware must be placed first to catch unhandled errors from downstream components.

10. How does Model Binding resolve parameters from different parts of an HTTP request?

- **Answer**:
- `[FromBody]` binds from the JSON/XML request body stream.
- `[FromRoute]` extracts values from the URL path template.
- `[FromQuery]` extracts key-value pairs from the query string.
- `[FromHeader]` reads HTTP request header values.
- `[FromForm]` parses multi-part form-data (including file uploads).

> **SkillHub Enterprise Architecture Specification Document**
> *Generated for SkillHub Engineering Documentation, Technical Audits, and Viva Evaluations.*